Project Report — CPU and Memory Stress Testing Tool (StressLab)

1. Title

CPU and Memory Stress Testing Tool — a controlled workload generator that stresses CPU and memory and measures system performance under different loads.

2. Abstract

StressLab is a complete web application that lets a user generate a controlled CPU + memory workload, watch live system performance, and compare runs. The user configures CPU workers, CPU intensity, memory size, memory pattern, duration, and load pattern (Sustained / Ramp / Spike / Step). The backend spawns worker threads, samples the operating system every second, streams results to the browser via Server-Sent Events, saves every run to SQLite, and enforces safety caps with a watchdog and one-click STOP. The frontend is a React dashboard with small minimal charts, per-core CPU bars, run detail views, presets, run comparison, and CSV export.

3. Problem Statement

Developers and students need to answer: how does a machine behave at 50% CPU vs 95%? At what memory usage does it slow down? How long can it sustain a spike? Existing tools (`stress-ng`, prime loops, ad-hoc scripts) are terminal-based, hard to control precisely, hard to observe (scrolling numbers, no history), and risky — a runaway allocation can freeze the machine. There is no simple, safe, web-based tool that combines controlled generation + live visibility + comparison + guardrails.

4. Objectives

1. Controlled generation: workers, intensity, memory size/pattern, duration, load pattern.
2. Live measurement: CPU (total + per-core), memory, swap, load average, throughput every second.
3. Safety by design: caps, watchdog, emergency STOP, clean worker termination.
4. Comparison & reporting: saved history, A/B compare, CSV export, copyable summaries.
5. OS learning value: make scheduling, paging, saturation, and thrashing visible.

5. Scope

In scope: web dashboard (React) + backend API (Node + Express), CPU stress via worker threads, memory stress via controlled allocations, real-time metrics via SSE, run history (SQLite), compare, CSV export, safety limits. Out of scope (deliberate): disk / network / GPU stress, distributed multi-machine stress, kernel tracing (eBPF/perf), multi-user accounts.

6. System Design

6.1 Architecture

Three layers — Frontend (React + Vite) ⇄ Backend (Node + Express, REST + SSE) → Stress Engine (worker_threads + controlled allocation) → Metrics + Store (os module + SQLite). The frontend never stresses the browser; all load is generated by the backend on the host, so measurements reflect the real OS. Full diagram and component table: `docs/ARCHITECTURE.md`.

6.2 Key design decisions

- **worker_threads for CPU:** fast to start, terminate cleanly with `worker.terminate()`, one worker per requested core.
- **Duty-cycle intensity:** to hit N%, a worker busy-loops N% of each 100ms window and sleeps the rest. Predictable average load without pegging the core.
- **Memory via Buffers + page-touching:** the worker writes 1 byte per 4KB page so the OS actually commits RAM. Plain allocation alone uses lazy pages and would not pressure memory.
- **SSE for live metrics:** one-way server → browser, simpler than WebSockets, auto-reconnects.
- **Single active run:** one run at a time keeps measurements interpretable and prevents accidental double-load. A second start returns 409.
- **Caps + watchdog:** memory ≤ 50% of RAM, workers ≤ cores, duration 5–600s. Watchdog auto-stops if free memory < 120 MB or memory use > 97%. Absolute thresholds — an early version used >92% used and false-triggered on a busy VM whose baseline was already 88% used. CPU 100% is intentional and does not auto-stop.
- **SQLite via Node's built-in `node:sqlite`:** zero-setup, no native dependency, DB at `backend/data/stress.db` (gitignored). Samples carry per-core data as `per_core_json`.

6.3 API

`GET /api/health`, `GET /api/system`, `POST /api/runs`, `GET /api/runs`, `GET /api/runs/:id`, `POST /api/runs/:id/stop`, `GET /api/runs/:id/stream` (SSE), `GET /api/runs/:id/export` (CSV). Full contract: `docs/ARCHITECTURE.md §3`.

6.4 Load patterns

Sustained (constant), Ramp (linear 0 → target), Spike (10% baseline with a burst to target in the middle), Step (three rising thirds: 33% / 66% / 100% of target).

7. Implementation

Backend (`backend/`, Node + Express)

- `src/server.js` — Express app, CORS, JSON, graceful shutdown (stops any active run on SIGINT/SIGTERM).
- `src/routes/system.js` — live host info from `os` module.
- `src/routes/runs.js` — run CRUD, STOP, SSE stream (replays past samples, then live), CSV export.
- `src/lib/stressEngine.js` — owns the single active run, spawns workers, 1s sampling loop, pattern intensity updates, watchdog, summary computation, EventEmitter for SSE.
- `src/lib/metrics.js` — CPU % from `os.cpus()` tick deltas (total + per-core), memory from `os.totalmem()/freemem()`, swap from `/proc/meminfo` on Linux.
- `src/lib/store.js` — SQLite persistence, runs + samples tables, per-core migration for older DBs.
- `src/lib/validate.js` — config validation with clear messages and host-aware caps.
- `src/workers/cpuWorker.js` — duty-cycle busy loop (sqrt work), reports ops, accepts live intensity updates.
- `src/workers/memoryWorker.js` — allocates Buffers, touches pages, hold/churn patterns, frees on STOP.

Frontend (`frontend/`, React + Vite)

- `src/App.jsx` — dashboard: System panel, New-run form + presets, Live metrics, Run detail, Compare, History.
- `src/lib/api.js` — REST helpers + EventSource wiring.
- `src/lib/presets.js` — 7 presets (Baseline, Light, Medium, Heavy, Spike, Ramp, Memory-pressure), auto-clamped to host cores/RAM.
- `src/lib/compare.js` — pure functions: merge samples by time, build compare CSV, run/compare summary text (unit-tested).
- `src/components/CompareChart.jsx` — overlay chart for two runs.
- Charts are pure SVG, 160px high, quiet gridlines, hover crosshair + tooltip. No chart library. Per-core bars, live stat tiles, progress bar with elapsed/remaining.

8. Testing & Validation

Full matrix and results: `docs/TESTING.md`. Summary of Phase 5 live results (2-core VM, 7935 MB RAM):

Test	Result
Sustained 1×50%, 128 MB, 8s	Completed, avg CPU 42.0%, 8 samples
Ramp 2×80%, 256 MB, 10s	Completed, avg 58.4%, 2nd half (60.2%) > 1st half (56.5%) — rising as configured
Spike 2×80%, 256 MB, 12s	Completed, middle 79.7% vs edges 34.5% — burst exactly as configured
Step 2×60%, 128 MB, 12s	Completed, thirds 35.3 / 48.1 / 66.1 — clean staircase
Zero-memory 1×50%, 0 MB, 5s	Completed, 5 samples
Max workers 2×100%, 0 MB, 5s	Completed, avg CPU 77.0% — highest in matrix, as expected
Churn 1×30%, 256 MB churn, 8s	Completed, 8 samples
STOP	0.08s to stopped
Concurrent run	409 run_active
Validation (7 bad configs)	All 400 with clear messages
SSE	5 sample events for a 5s run
Compare (Light vs Medium, live)	A 34.0% vs B 64.1%, diff +30.2pp, merged 8 rows, CSV 9 lines

Honest limitations: CPU % is total system CPU, so background processes add noise — comparisons on the same machine are more meaningful than absolute numbers. Ramp separation was compressed by background load on the shared test VM; Spike and Step shapes

were unambiguous. The watchdog's live OOM path was deliberately not triggered (it would risk freezing the test VM); it was verified by code review, by the earlier false-trigger fix, and by normal runs not triggering it.

9. Results

- A working prototype that fulfils the problem statement: controlled generation, live measurement, safety, history, compare, export.
- Patterns produce their configured shapes (Spike and Step unambiguously; Ramp directionally).
- Heavier configs produce higher measured CPU (Light 34.0% vs Medium 64.1% vs Max 77.0%).
- STOP is sub-second; memory is freed after every run (verified: 580 → 268 → 555 MB around a 512 MB run).
- All validation, concurrency, SSE, persistence, and migration tests pass; frontend build passes with zero errors.

10. Business Impact / Applications

- **Students / OS learners:** see scheduling, per-core parallelism, paging, saturation, and thrashing live.
- **Developers / QA:** check how an app behaves when the machine is busy (e.g. run a test suite under 70% CPU).
- **DevOps / capacity planning:** validate a laptop, VM, or small server before deploying; compare configs before buying bigger hardware.
- **Hiring / demos:** a safe, visual way to show OS concepts without terminal expertise.

11. Originality & Responsible Implementation

Original implementation written for this hackathon. Runs locally in a controlled environment, single active run, capped loads, watchdog + STOP, no disruption to other users or system processes. No code was copied from existing stress tools; the duty-cycle, page-touching, and pattern logic were designed from OS principles and verified by measurement.

12. Future Work

Disk / network / GPU stress, multi-machine stress, per-process attribution (which process used the CPU), scheduled runs, and PDF report export.

13. Conclusion

StressLab delivers what the title asks for: a controlled workload generator that stresses CPU and memory and measures system performance under different loads — as a complete, working web application with safety, history, comparison, and documentation.